



DATA ENGINEER CHALLENGE

Deliverable A

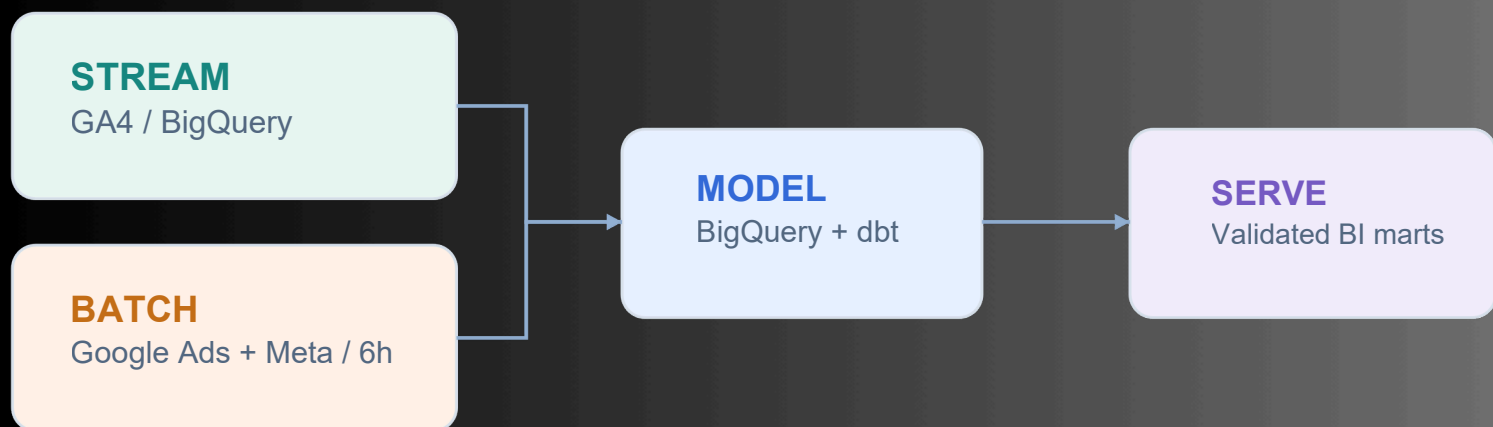
GCP Architecture Design

A decoupled and scalable architecture for GA4 streaming and Ads batches every six hours.

Prepared by

Andres Olguin

Data Engineer



01 Executive Summary

A theoretical GCP design that preserves fresh GA4 analytics while Ads batches arrive and reconcile independently.

Architecture intent

GA4 native streaming and daily exports land in BigQuery. Google Ads and Meta Ads files land in Cloud Storage every six hours. Independent Cloud Run Jobs validate and load Ads; a separate job runs dbt in BigQuery. BI reads the latest validated publication, including clearly labeled provisional windows.

Decoupled by source

Ads ingestion failures do not stop GA4 ingestion or its independent analytical publication.

Idempotent by design

Version-aware loads and canonical keys make retries repeatable without multiplying spend.

Event-time aware

Landing event time determines session attribution. Processing checkpoints track operational progress.

Validated and recoverable

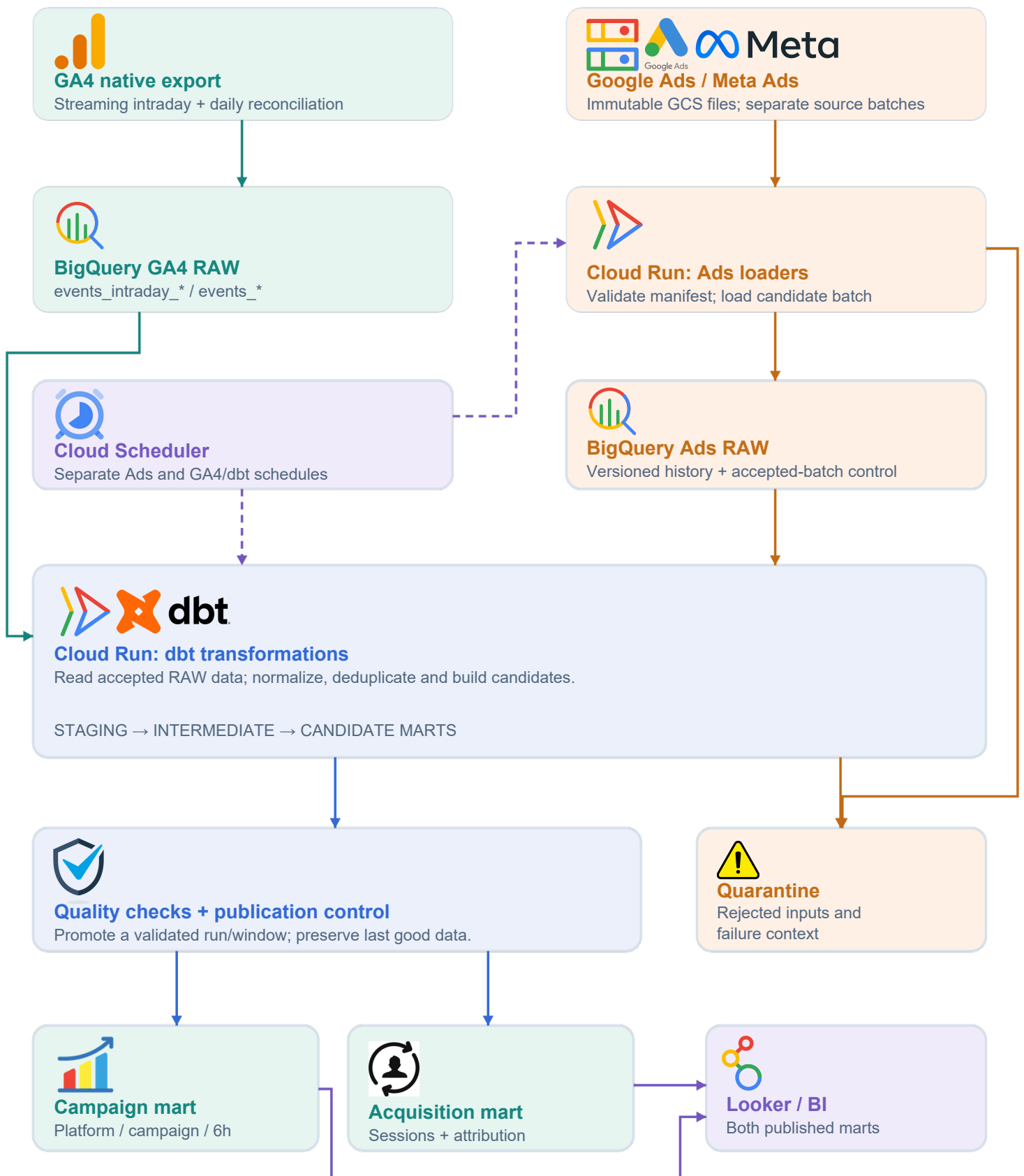
Candidate outputs pass checks before promotion. Failures retain the last successful publication.

Scope and explicit assumptions

- Source boundary: GA4 export and delivery of Ads files to GCS are upstream inputs. API extraction from ad platforms is outside this design; the loading and reconciliation path is in scope.
- Batch contract: each version is a complete replacement snapshot for one source and six-hour window, at campaign grain. Objects are immutable; corrections create a higher version.
- Time and money: UTC is the proposed shared window timezone, subject to source-owner agreement. Inputs must use a common currency or an explicit conversion policy before ROI is compared.
- Execution: dbt runs in Cloud Run. GA4 processing and Ads discovery have separate schedules. GCP deployment is not required; Deliverable B demonstrates the lab semantics in Postgres.

02 Reference Architecture

Solid arrows carry data; dashed arrows trigger work. RAW boxes are the same source datasets read by dbt.



Across all stages: IAM • Cloud Logging / Monitoring • run and window metadata • backfill audit

03 Ingestion and Processing

Source-specific ingestion converges into canonical records. A native export adapter bridges GA4 and the analytical contract.

GA4: native export adapter

Streaming: read current-day intraday tables for low-latency, provisional results. Native streaming is best effort, not a completeness guarantee. [1]

Daily reconciliation: revisit the current date and three preceding dates. Prefer daily tables when available; do not blindly union daily and intraday copies. [2]

Canonical contract: normalize user/session IDs, event time, campaign attribution and purchase values. Native GA4 does not provide the emulator's ingested_at field.

Progress: record successful source-table scans and observation times. Rescan mutable dates; a newly assigned observed_at cannot detect changes that were never scanned.

Ads: six-hour snapshots

Input: immutable source/date/window/version objects plus a manifest identifying expected files and rows. Empty valid batches are explicit.

Validation: check schema, window, currency, business-key uniqueness and manifest totals before accepting a batch.

Control: persist object URI/generation, checksum, version, window, run ID, counts and status. Retain history; expose only accepted versions.

Correction: replace the full source-window snapshot, including removal of rows absent from the new version. Reject stale versions; never append revised spend as new activity.

Keys, grain and canonicalization

Dataset	Contract
GA4 events	Prefer a stable producer event ID where available. Otherwise define and validate a composite from property/stream, user, timestamp, event name and bundle/batch sequence fields; timestamp alone is insufficient. Preserve legitimate repeated events and deterministic source precedence.
Ads snapshots	Challenge grain: platform + campaign_id + window_start. Campaign IDs are unique within each source in the lab. Extend the key with account_id for production accounts if IDs can collide.
Session attribution	Use the selected landing event for both campaign_id and landing_timestamp. The lab prioritizes campaign-bearing landing events, then the earliest timestamp. A native GA4 adapter must define the equivalent landing/campaign mapping.

Physical design: partition canonical event/batch tables by business date; cluster on frequently filtered keys. Require partition filters and bounded scans. Materialize expensive session aggregates as volume grows.

04 Streaming-Batch Coupling

Availability, operational progress and business finalization are distinct. Fresh results remain revisable.

State / signal	Meaning and publication behavior
pending_ads	GA4 campaign activity exists, but its matching Ads data is unavailable. Publish GA4 metrics; keep spend and dependent ratios NULL.
ads_available	Matching Ads data is available. Reconcile without multiplying spend. This is not a claim that GA4 or the whole window is complete.
Operational finalization	Finalize under an explicit policy only after expected source manifests pass and GA4 daily reconciliation covers the agreed lateness horizon. Retain an as-of timestamp; corrections can reopen the window.

Two time axes and a completeness policy

- Event time: determines session attribution and campaign-window membership. Use UTC windows [00:00,06:00), [06:00,12:00), [12:00,18:00), [18:00,24:00). Exactly 06:00 belongs to the second window.
- Processing checkpoint: records successfully processed inputs per source. Commit it only after the corresponding canonical write succeeds. It is not an event-time completeness watermark.
- Late data: for an ingestion-timestamp source, an old event with a new ingestion time is picked up normally; overlap protects run boundaries. For native GA4, rescan mutable daily tables and reconcile changes.
- Finalization policy: require accepted manifests for the expected Ads sources/accounts and completed GA4 reconciliation through D+3 for event date D. This is an operational cutoff, not proof that future corrections are impossible.

A late event can affect an earlier window

Detect changed events, identify their sessions, and recompute each affected session from its required history. A purchase may update the earlier landing window; a changed landing may move attribution. Rebuild both old and new affected windows, not only the arrival-date partition.

Beyond the automatic horizon: open an audited backfill for named dates/sessions, reconcile, validate and republish. Missing data never becomes zero merely because a deadline has passed.

05 Reliability and Safe Publication

At-least-once triggers are expected. Input acceptance, canonicalization and BI publication have separate success boundaries.

Failure / risk	Response
Duplicate or concurrent run	Serialize writers per source-window; use a deterministic batch/run identity and version checks. Check control state before retrying and reuse a deterministic BigQuery load-job identity. Commit accepted state only after the data write succeeds.
Malformed data / schema drift	Reject incompatible batches to restricted quarantine with object and row context. Preserve RAW and the prior accepted version. GA4 validation rejects are also observable; missing Ads must not block independent GA4 processing.
Transient failure / late file	Retry transient failures with bounded exponential backoff. Poll unaccepted manifests, including overdue windows, rather than checking once every six hours. Alert after the retry or lateness threshold.
Failed dbt model / test	Retain candidate outputs and run artifacts for diagnosis. Do not advance the published pointer. Monitor actual Cloud Run execution completion, not only successful Scheduler dispatch.

Build, validate, then promote

- 1. Scope: resolve accepted input versions and affected sessions/windows. Record the run manifest; keep immutable source history separate from canonical outputs.
- 2. Build: write a candidate version of affected outputs without changing published data. Recompute full affected sessions and remove superseded window rows.
- 3. Validate: check keys, bounds, source-to-mart spend reconciliation, session preservation, NULL ratios and late-event propagation. dbt build alone is not a transaction across the entire publication. [3]
- 4. Promote: after checks pass, atomically update publication metadata selecting the accepted run for each affected scope. BI views resolve only accepted versions. Update coupled marts in one publication transaction.
- 5. Recover: on failure, retain the last good version with its as-of time; retry safely. Keep run, input-version and publication history. Separate GA4-only publications from Ads-dependent scopes.

Backfill uses the same path: explicit date/session scope → canonical rebuild → candidate marts → reconciliation → promotion. It bypasses the normal incremental filter for the selected scope.

06 Operations, Security and Scale

Managed services keep the design small. Explicit ownership, freshness targets and scan limits make it operable.

Observability

Log run ID, source, window, object version, accepted/rejected rows and final execution status.

Alert on failures, missing batches, stale publications, rejection spikes and spend mismatches.

Proposed targets: poll every 5 min; publish within 15 min of source availability; alert when Ads is 30 min overdue. Validate these assumptions with the source owner.

Security

Separate identities for job invocation, RAW loading, dbt candidates, publication and BI reads.

Use least privilege, managed identity, Secret Manager and encryption at rest/in transit.

Restrict user IDs, URLs and quarantine. Agree retention/deletion rules and compatible GCS/BigQuery locations.

Scalability

Partition pruning and bounded incremental scans control BigQuery cost. Materialize costly session aggregates when justified.

Parallelize independent source windows; serialize conflicting writers. More dbt workers do not guarantee faster SQL.

Monitor query bytes, duration and cost. Set budgets and concurrency limits; expire candidates under an agreed audit policy.

Design decisions and trade-offs

Choice	Benefit and accepted cost / upgrade trigger
Cloud Run + Scheduler	Chosen: simple independent schedules. Costs: polling latency, retry/control logic and protection from overlapping runs. Scheduler delivers at least once. [4]
Native GA4 export	Chosen: managed ingestion without a custom event bus. Costs: best-effort freshness, mutable daily data and an explicit schema/attribution adapter.
Microbatch dbt + BigQuery	Chosen: reusable SQL and bounded recomputation. Costs: scan spend and minutes of publication latency. Candidate versions add storage in exchange for safe promotion.
Pub/Sub + Dataflow	Deferred. Consider for an upstream custom event feed needing fan-out, stateful streaming or sub-minute processing. These do not remove native GA4 export latency.
Workflows / Composer	Deferred. Add orchestration when cross-job dependencies and recovery complexity exceed the small independently scheduled design.

No fixed throughput claim: size slots, job resources and retention from measured source volume, query cost and the agreed freshness objective.

07 Consumption and Challenge Coverage

Published marts connect the architecture to the business questions; the lab validates a deliberately narrower implementation.

Published model	Grain and business use
Campaign performance	Platform + campaign + six-hour window. Aggregate GA4 sessions first, then combine with Ads at the same grain to avoid spend fan-out. Preserve Ads-only and GA4-only windows. Supports platform efficiency and campaign ranking.
Acquisition performance	One row per user/session with attribution status, conversions, purchases and revenue. Preserve unattributed sessions. Combine paid-channel aggregates with campaign-mart spend at compatible grain; never repeat spend per session.

Metric and reporting contract

CPC = spend / clicks; CPA = spend / source-defined conversions; ROI = (purchase revenue - spend) / spend. Compute ratios from aggregated totals, not averages of row ratios; missing or zero denominators yield NULL. ROI is an advertising-return proxy, not net profit. Show data status and as-of time. Unattributed does not mean organic/direct; full channel reporting needs source/medium data and a defined mapping.

What B demonstrates; what A proposes

Demonstrated in B: Postgres staging uses incremental processing with a one-minute ingestion lookahead; downstream models are views. The implementation preserves pending Ads and unattributed sessions, aligns campaign windows and tests reconciliation. Its late-arrival test checks unique presence of late keys in staging.

Designed for A: the native GA4 adapter, versioned GCS ingestion, source manifests, concurrency control, targeted BigQuery backfills and candidate-to-published promotion. These require GCP implementation and operational validation; B does not claim to prove them.

Coverage: heterogeneous ingestion (pp. 3-4); coupling, lateness and progress (p. 5); errors, idempotence and recovery (p. 6); scalable operations (p. 7); analytical consumption (p. 8).

References and delivery basis

[1] [Google Analytics: BigQuery Export](#)

[2] [Google Analytics: BigQuery Export schema](#)

[3] [dbt: build command](#)

[4] [Google Cloud: Cloud Scheduler overview](#)

Basis: original XYZ x .Monks challenge PDF, repository ENUNCIADO.md and local Deliverable B models/tests. Reference review: 18 September 2026. Theoretical design; no GCP deployment claimed.